

autumnqp 님의 블로그

Search...

메뉴바를 클릭하면
계정을 관리할 수 있어요

카테고리 없음

[Paper Review] Attention Is All You Need (Transformer)

autumnqp 2026. 5. 6. 20:03

논문 링크

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. NeurIPS 2017.
<https://arxiv.org/abs/1706.03762>

1. Introduction — 왜 RNN을 버렸는가?

이전 리뷰들에서 다룬 Seq2Seq, Bahdanau Attention은 모두 **RNN(또는 LSTM)** 을 기반으로 했습니다. RNN 기반 모델은 시퀀스를 왼쪽에서 오른쪽으로 **순서대로** 처리해야 한다는 구조적 특성이 있었어요.

이 순차적 구조에는 두 가지 치명적인 문제가 있습니다.

병렬화 불가: t 번째 은닉 상태 h_t 를 계산하려면 반드시 h_{t-1} 이 먼저 계산되어 있어야 합니다. 즉 GPU가 아무리 많아도 한 문장을 처리할 때는 순서대로 기다려야 하므로 병렬화가 근본적으로 불가능합니다.

장거리 의존성 한계: 문장이 길어질수록 멀리 떨어진 단어들 사이의 관계를 학습하기 어렵습니다. Bahdanau Attention이 이를 많이 완화했지만, RNN 자체의 구조적 한계는 여전히 남아 있습니다.

이 논문은 RNN을 **완전히 제거**하고, **Attention만으로** 인코더와 디코더를 구성하는 Transformer 아키텍처를 제안합니다. 핵심 주장은 단순합니다. "Attention만으로도 충분하다 (Attention Is All You Need)."

결과적으로 Transformer는 WMT 2014 영어-독일어 번역에서 28.4 BLEU, 영어-프랑스어 번역에서 41.8 BLEU를 달성하며 당시 모든 모델을 뛰어넘었고, 학습 시간은 기존 최고 모델의 극히 일부에 불과했습니다.

2. Background — RNN 대안 탐색의 역사

RNN의 순차 처리 문제를 해결하려는 시도는 이전에도 있었습니다. Extended Neural GPU, ByteNet, ConvS2S 등은 CNN을 기본 구성 요소로 사용해 병렬 처리를 시도했습니다. 하지만 CNN 기반 모델에서는 두 위치 사이의 관계를 계산하는 데 필요한 연산 수가 위치 간 거리에 비례해 늘어납니다(ConvS2S는 선형, ByteNet은 로그). 거리가 멀수록 학습이 어려워지는 문제가 여전히 남아 있었죠.

Self-Attention 은 이미 다양한 태스크에서 사용되고 있었지만, 항상 RNN과 함께 보조적으로 사용됐습니다. Transformer는 Self-Attention만으로 RNN 없이 전체 모델을 구성한 첫 번째 시도입니다.

3. Model Architecture — Transformer 전체 구조

Transformer는 기존과 동일하게 **인코더-디코더** 구조를 따릅니다. 인코더가 입력 시퀀스를 연속 표현으로 변환하면, 디코더가 이를 받아 출력 시퀀스를 하나씩 생성합니다. 차이는 그 내부가

RNN이 아닌 **Self-Attention + Feed-Forward** 레이어로 이루어져 있다는 점입니다.

3.1 Encoder

인코더는 **동일한 레이어 6개(N=6)**를 쌓은 구조입니다. 각 레이어는 두 개의 서브레이어로 구성됩니다.

1. **Multi-Head Self-Attention**: 입력 시퀀스 내 모든 위치 간 관계를 계산
2. **Position-wise Feed-Forward Network**: 각 위치마다 독립적으로 적용되는 2층 신경망

각 서브레이어에는 **잔차 연결(Residual Connection)** 과 **레이어 정규화(Layer Normalization)** 가 적용됩니다.

$$\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

모든 서브레이어의 출력 차원은 $d_{\text{model}} = 512$ 로 통일되어 있어 잔차 연결이 자연스럽게 적용됩니다.

3.2 Decoder

디코더도 6개 레이어를 쌓은 구조이지만, 인코더와 달리 **세 개의 서브레이어**를 가집니다.

1. **Masked Multi-Head Self-Attention**: 디코더 내부의 자기 주의. 현재 위치보다 뒤에 있는 토큰을 미래 정보를 보지 못하도록 **마스킹($-\infty$)** 처리합니다.
2. **Multi-Head Cross-Attention**: 디코더의 쿼리가 인코더 출력 전체를 참조합니다. Bahdanau Attention과 기능적으로 같은 역할이에요.
3. **Position-wise Feed-Forward Network**

마스킹의 이유: 번역 시 i 번째 단어를 예측할 때 $i + 1$ 번째 이후 정보를 사용하면 "답지를 보고 시험 치는" 격이 됩니다. 학습 시 이를 방지하기 위해 미래 위치를 $-\infty$ 로 마스킹하여 소프트맥스 후 가중치가 0이 되게 합니다.

4. Attention 메커니즘 — 핵심 수학

4.1 Scaled Dot-Product Attention

Transformer의 Attention은 **Query(Q), Key(K), Value(V)** 세 가지 행렬로 작동합니다. 먼저 각각의 역할을 직관적으로 이해해 볼게요.

도서관 비유를 들면, Query는 "내가 찾는 책의 주제", Key는 "각 책의 제목(색인)", Value는 "실제 책의 내용"입니다. Query와 Key를 비교해서 얼마나 관련 있는지(가중치)를 구하고, 그 가중치로 Value를 가중합하면 "내 질문에 가장 관련 높은 내용을 담은 답"이 됩니다.

수식으로는 다음과 같습니다.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^{\top}}{\sqrt{d_k}} \right) V$$

$\sqrt{d_k}$ 로 나누는 이유가 중요합니다. d_k 가 크면 $QK^{\top}$ 내적값이 커져서 소프트맥스 함수가 기울기가 거의 0인 포화(saturation) 영역에 빠집니다. 이를 방지하기 위해 $\sqrt{d_k}$ 로 스케일링합니다. 이것이 기존 Bahdanau의 Additive Attention과의 핵심 차이점이기도 합니다.

Bahdanau Attention은 $e_{ij} = v_a^{\top} \tanh(W_a s_{i-1} + U_a h_j)$ 처럼 피드포워드 신경망으로 점수를 계산했지만, Scaled Dot-Product는 행렬 곱으로 계산하기 때문에 **훨씬 빠르고 메모리 효율적**입니다.

4.2 Multi-Head Attention

단 하나의 Attention 함수로 Q, K, V를 처리하는 대신, h 개의 헤드로 나누어 **병렬로 여러 번의 Attention을 수행**하고 결과를 이어붙입니다.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

왜 여러 헤드가 필요할까요? 하나의 Attention은 특정 유형의 관계(예: 구문적 관계)에만 집중하는 경향이 있습니다. 여러 헤드를 사용하면 각 헤드가 서로 다른 관점(구문, 의미, 지시 관계 등)에서 단어 간 관계를 동시에 학습할 수 있습니다.

이 논문에서는 $h = 8$ 개의 헤드를 사용하고, $d_k = d_v = d_{\text{model}}/h = 64$ 로 설정합니다. 헤드별로 차원이 줄어드므로 전체 연산량은 단일 헤드 full-dimension Attention과 유사합니다.

4.3 Transformer 내 Attention의 세 가지 사용 방식

Transformer는 Attention을 세 가지 다른 방식으로 사용합니다.

① **인코더 Self-Attention**: Q, K, V 모두 인코더 이전 레이어 출력에서 옵니다. 인코더의 각 단어가 같은 문장 내 모든 다른 단어를 참조합니다.

② **디코더 Masked Self-Attention**: Q, K, V 모두 디코더 이전 레이어 출력에서 오지만, 미래 위치는 마스킹됩니다.

③ **디코더 Cross-Attention**: Q는 디코더에서, K와 V는 인코더 최종 출력에서 옵니다. 이것이 Bahdanau Attention과 기능적으로 대응되는 부분입니다.

5. Position-wise Feed-Forward Network

각 레이어에는 Attention 서브레이어 외에도 **FFN(Feed-Forward Network)** 이 있습니다.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

입력/출력 차원은 $d_{\text{model}} = 512$, 내부 은닉층 차원은 $d_{\text{ff}} = 2048$ 입니다. 같은 레이어 내에서는 모든 위치에 동일한 파라미터를 적용하지만, 레이어마다 파라미터는 다릅니다. 이 FFN은 Attention이 "어디서 정보를 가져올지" 결정한다면, FFN은 "가져온 정보를 어떻게 변환할지"를 담당하는 역할을 합니다.

6. Positional Encoding — 순서 정보를 어떻게 넣나?

RNN은 시퀀스를 순서대로 처리하면서 자동으로 위치 정보가 인코딩됩니다. 하지만 Transformer는 모든 위치를 동시에 처리하기 때문에 **위치 정보를 별도로 주입**해야 합니다. 이 논문은 사인/코사인 함수를 이용한 **Sinusoidal Positional Encoding**을 제안합니다.

$$PE_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{2i/d_{model}}} \right)$$

$$PE_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{2i/d_{model}}} \right)$$

여기서 pos 는 토큰의 위치, i 는 임베딩의 차원 인덱스입니다.

이 방식을 선택한 이유는 두 가지입니다. 먼저 임의의 고정 오프셋 k 에 대해 PE_{pos+k} 가 PE_{pos} 의 선형 변환으로 표현될 수 있어, 모델이 **상대적 위치 관계**를 쉽게 학습할 수 있습니다. 또한 학습 데이터에서 보지 못한 더 긴 시퀀스 길이로 **외삽(extrapolation)**이 가능합니다. 실험적으로는 학습된 positional embedding과 성능 차이가 거의 없었습니다 (Table 3 row E).

7. Why Self-Attention? — RNN, CNN과의 비교

왜 Self-Attention이 RNN, CNN보다 우수한지를 세 가지 기준으로 비교합니다.

① 레이어당 연산 복잡도

레이어 타입 복잡도 순차 연산 최대 경로 길이

Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$

② **병렬화**: Self-Attention은 모든 위치를 한 번에 처리하므로 순차 연산 수가 $O(1)$ 입니다.

RNN은 $O(n)$ 으로 문장이 길수록 병목이 심해집니다.

③ **장거리 의존성**: 임의의 두 위치 사이의 최대 경로 길이가 Self-Attention은 $O(1)$ 입니다. 어떤 두 단어도 단 한 번의 Attention 연산으로 직접 연결됩니다. RNN은 $O(n)$ 으로 멀리 떨어진 단어일수록 신호가 희석됩니다.

단, Self-Attention은 시퀀스 길이 n 이 표현 차원 d 보다 클 때 연산량이 RNN보다 커질 수 있습니다. 이를 위해 논문은 제한된 범위의 Self-Attention(restricted self-attention)을 미래 작업으로 언급합니다.

8. Experiment Settings

- **데이터셋**: WMT 2014 영어-독일어 (450만 문장 쌍), 영어-프랑스어 (3,600만 문장 쌍)
- **토큰나이징**: Byte-Pair Encoding (BPE), 영-독 공유 어휘 37,000개, 영-프 32,000 word-piece
- **하드웨어**: 8× NVIDIA P100 GPU
- **학습 시간**: Base 모델 12시간(100K 스텝), Big 모델 3.5일(300K 스텝)
- **옵티마이저**: Adam ($\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 10^{-9}$) + Learning Rate Warmup

$$lr_{rate} = d_{model}^{-0.5} \cdot \min(\text{step_num}^{-0.5}, \text{step_num} \cdot \text{warmup_steps}^{-1.5})$$

처음 $warmup_steps = 4000$ 스텝 동안은 학습률을 선형으로 증가시키고, 이후에는 스텝의 역제곱근에 비례해 감소시킵니다. 이 Warmup 스케줄은 이후 BERT 등에서도 널리 채택됩니다.

- **정규화**: Residual Dropout ($P_{drop} = 0.1$), Label Smoothing ($\epsilon_{ls} = 0.1$)
- **디코딩**: 빔 서치 (빔 크기 4, 길이 패널티 $\alpha = 0.6$)
- **모델 크기**:

N d_{model} d_{ff} h d_k d_v 파라미터

Base	6	512	2048	8	64	64	65M
Big	6	1024	4096	16	64	64	213M

9. Results

9.1 기계 번역 성능

주요 결과를 요약하면 다음과 같습니다.
모델 EN-DE BLEU EN-FR BLEU 학습 비용

GNMT + RL Ensemble	26.30	41.16	1.8×10^{20}
ConvS2S Ensemble	26.36	41.29	7.7×10^{19}
Transformer (base)	27.3	38.1	3.3×10^{18}
Transformer (big)	28.4	41.8	2.3×10^{19}

Transformer (big)은 영-독에서 기존 양상블 최고 모델을 **2 BLEU 이상** 앞질렀습니다. 영-프에서는 41.8 BLEU로 단일 모델 SOTA를 달성했습니다. 특히 학습 비용(FLOPs)을 보면 Transformer (base)는 ConvS2S Ensemble보다 **약 23배 적은 연산량**으로 더 높은 성능을 냈습니다.

9.2 모델 변형 실험 (Ablation Study)

Table 3의 핵심 결과를 정리하면 다음과 같습니다.

(A) 헤드 수 변화: 단일 헤드(1개)보다 8개가 최적이었고, 너무 많은 헤드(32개)도 성능이 떨어졌습니다. 헤드당 차원 d_k 가 너무 작아지면 성능이 저하됩니다.

(B) d_k 감소: d_k 를 줄이면 성능이 하락합니다. 단순한 Dot-Product로 호환성을 계산하는 것이 충분하지 않을 수 있음을 시사합니다.

(C) 모델 크기: 모델이 클수록 성능이 좋아지는 경향이 일관되게 나타납니다.

(D) 드롭아웃: 필수적입니다. 드롭아웃 없이는 과적합이 발생합니다.

(E) Positional Encoding: Sinusoidal vs. 학습된 임베딩 — 성능 차이가 거의 없습니다 (25.8 vs 25.7).

9.3 영어 구문 분석(Constituency Parsing)으로의 일반화

번역 외 다른 태스크에서도 Transformer가 잘 작동하는지 검증했습니다. WSJ Penn Treebank 영어 구문 분석 태스크에서 4-layer Transformer가 대부분의 기존 모델을 능가했습니다. 태스크 특화 튜닝 없이 번역 모델 하이퍼파라미터를 거의 그대로 사용했다는 점에서 Transformer의 범용성을 보여줍니다.

10. Attention 시각화 — 모델이 무엇을 배웠나?

논문 부록에는 Attention 가중치 시각화 결과가 있습니다. 서로 다른 헤드들이 각기 다른 언어적 관계를 학습했음을 보여줍니다. 예를 들어 어떤 헤드는 주어-동사 관계를, 다른 헤드는 한정사-명사 관계를 학습합니다. 이는 Multi-Head Attention이 단순히 여러 번 Attention을 수행하는 것을 넘어, **다양한 언어 구조를 동시에 학습**한다는 것을 직관적으로 보여줍니다.

11. 결론 및 의의

이 논문은 NLP 역사에서 가장 영향력 있는 논문 중 하나로, 여러 측면에서 패러다임을 바꾸었습니다.

RNN의 완전한 대체: Seq2Seq → Bahdanau Attention으로 발전하면서도 RNN은 항상 핵심 구성 요소로 남아 있었습니다. Transformer는 RNN을 완전히 제거하고 Attention만으로 동등 이상의 성능을 달성함으로써 RNN 시대의 종말을 알렸습니다.

병렬화를 통한 스케일링: RNN의 순차 처리 한계가 사라지면서 훨씬 더 큰 모델을 훨씬 더 빠르게 학습할 수 있게 되었습니다. 이는 이후 GPT, BERT 등 수억~수천억 파라미터 모델의 등장을 가능하게 한 직접적인 토대가 됩니다.

범용 아키텍처의 탄생: Transformer는 기계 번역을 위해 설계됐지만, 이후 텍스트(BERT, GPT), 이미지(ViT), 오디오(Whisper), 멀티모달(DALL-E, GPT-4V) 등 거의 모든 딥러닝 분야로 확산됩니다. 오늘날 우리가 사용하는 ChatGPT, Claude를 포함한 모든 대형 언어 모델의 근간이 바로 이 논문의 아키텍처입니다.

지금까지 리뷰한 논문들을 흐름으로 정리하면 다음과 같습니다.

Seq2Seq (2014) — RNN으로 가변 길이 번역 최초 구현 → **Bahdanau Attention (2015)** — 고정 벡터 병목 해결, 소스 동적 참조 → **ELMo (2018)** — 사전 학습 표현의 힘 증명 → **Transformer (2017)** — RNN 제거, Attention만으로 SOTA 달성 → **BERT, GPT, ...** — Transformer를 대규모로 사전 학습

'카테고리 없음'의 다른글

이전글 [Paper Review] Deep Contextualized Word Representations (ELMo)

현재글 : [Paper Review] Attention Is All You Need (Transformer)

다음글 [Paper Review] Improving Language Understanding by Generative Pre-Training (GPT-1)

autumnqp 님의 블로그

autumnqp 님의 블로그 입니다.

댓글 0



autumnqp

등록



autumnqp님의 블로그

autumnqp님의 블로그입니다.

글쓰기

블로그 관리

분류 전체보기 (6) 

Paper review (0)

Tag

CLS, elmo, Convolutional Neural Networks, NLP,
biLM, DNNs, BERT, LLM, encoder-decoder,
Seq2Seq, gpt-1, Attention, NMT, RNN,
Deep contextualized word representations,
fine-tuning, CoVe, Transformer, Bleu, LSTM

최근글 인기글

[Paper Review] BERT: Pre-training of Deep
Bidirectional Transformers...

2026.05.06 22:36

[Paper Review] Improving
Language Understanding by
Generative Pre-Tr...

2026.05.06 21:45

[Paper Review] Attention Is All You
Need (Transformer)

2026.05.06 20:03

최근댓글

공지사항

Facebook Twitter

Archives

2026/05

2026/04

〈 2026. 05 〉						
일	월	화	수	목	금	토
					1	2
3	4	5	<u>6</u>	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31						

Total

4

Today : 1

Yesterday : 2

검색내용을 입력하세요.

Copyright © AXZ Corp. All rights reserved.

관련사이트